

CLASSIFICATION OF HAND-WRITTEN DIGITS

ÁLVARO GALISTEO BERMÚDEZ, DANIEL RATH

ABSTRACT. This report studies hand-written digit classification on the *sklearn* Digits dataset using multinomial logistic regression trained with gradient-based methods. We cast training as a finite-sum convex optimization problem and compare full-batch gradient descent (full-GD) against mini-batch stochastic gradient descent (SGD) in terms of objective decrease, validation/test accuracy, and wall-clock time.

Based on systematic hyperparameter sweeps, we observe that SGD reaches high validation accuracy in substantially less wall-clock time, while full-GD exhibits smoother, deterministic objective trajectories but incurs higher per-iteration cost. Both approaches achieve strong test performance, with SGD providing the best time-to-accuracy trade-off in our experimental setup.

This report documents the final project for the lecture *Basics in Applied Mathematics*, offered in the winter term 2025/2026 within the master's program *Mathematics in Data and Technology* at the University of Freiburg.

CONTENTS

1. Logistic regression for multi-class classification	1
1.1. Our problem	1
1.2. Loss function	2
1.3. Solvability	2
1.4. Regularization	3
2. Stochastic Gradient Descent	5
2.1. Gradient-based methods: cost–accuracy trade-offs	5
3. Implementation	7
3.1. Code structure	7
3.2. Data pipeline	7
3.3. Model, objective, and derivatives	7
3.4. Optimization algorithms	8
3.5. Experimental protocol	8
3.6. Evaluation metrics and visualizations	9
4. Results and Discussion	10
4.1. Hyperparameter tuning	10
4.2. Final models and discussion	12
References	14

1. LOGISTIC REGRESSION FOR MULTI-CLASS CLASSIFICATION

Logistic regression is a commonly used model for classification tasks. In our case we want to use logistic regression for multi-class classification, which is a generalization of the binary logistic regression.

In our case, we want to predict the class of a given 8×8 grayscale image of a hand written digit. One would expect that a given input can not be classified as more than one digit at the same time. Therefore, we want to compute the probability of a given input image to belong to every of the 10 classes (digits from 0 to 9) and assign the class with the highest probability to the input image.

1.1. Our problem.

An input image is represented as a vector $a_j \in \mathbb{R}^{64}$. Each entry of the vector is an integer between 0 and 16 and corresponds to the intensity of a pixel in the image with 0 being the lowest intensity and 16 being the highest intensity. Each image a_j in the dataset is associated with a label $y_j \in \{0, \dots, 9\}$, which is the digit that is represented in the image. In order to use the labels in our model, we encode them into one-hot vectors $p_j \in \{0, 1\}^{10}$, where the i -th entry is 1 if the label is i and 0 otherwise. [Dev]

We want to train a function $\varphi : \mathbb{R}^{64} \times (\mathbb{R}^{64} \times \mathbb{R})^{10} \rightarrow \mathbb{R}^{10}$ that takes an input image a_j and a parameter $x = (w_i, b_i)_{i=0}^9$ and outputs a vector of scores for each class. The score for class i is computed as $\varphi(a_j; x)_i = a_j^T w_i + b_i$. The $w_i \in \mathbb{R}^{64}$ is a weight vector and $b_i \in \mathbb{R}$ is a bias term for class i .

The training of the model consists of the following optimization problem

$$(1.1) \quad \underset{x \in (\mathbb{R}^{64} \times \mathbb{R})^{10}}{\text{minimize}} \quad \frac{1}{N} \sum_{j=1}^N L(p_j, \varphi(a_j; x)),$$

with $N \in \mathbb{N}$ being the number of training samples and L the cross entropy loss defined as

$$(1.2) \quad L(p, s) = \log \left(\sum_{l=0}^9 \exp(s_l) \right) - \sum_{i=0}^9 p_i s_i.$$

After the model has been trained, we can use it to classify new input images a by computing $\varphi(a; x)$ and assigning the class with the highest score to the input image, i.e.

$$\tilde{y} = \arg \max_{i \in \{0, \dots, 9\}} \varphi(a; x)_i.$$

For logistic regression the scores in $\varphi(a; x)_i$ are interpreted as probabilities by applying the softmax function, i.e.

$$(1.3) \quad \sigma(s)_i = \frac{\exp(s_i)}{\sum_{l=0}^9 \exp(s_l)}.$$

Note that in order to classify an input image, we do not need to compute the probabilities $\sigma(\varphi(a; x))_i$ explicitly. This is because the softmax function is monotonically increasing in each of its entries. Therefore, the class with the highest score is also the class with the highest probability. However, we will see in the next section that the softmax function is important for the choice of our loss function.

1.2. Loss function.

We want to briefly discuss the choice of the loss function (1.2). For this sections we used [GBC16, Section 6.2.2.3] as a reference. In general, for a given input image a_j we want to predict the probabilities $\sigma(\varphi(a_j; x))_i$ to belong to each of the classes $0 \leq i \leq 9$.

For this, we want our model to maximize the likelihood to observe the training data. If we assume each training image to be independent, the likelihood that our model predicts the observed training data correctly is given as

$$\mathcal{L}(x) := \prod_{j=1}^N \sigma(\varphi(a_j; x))_{y_j}.$$

In order to deal with sums instead of products, we can equivalently minimize

$$(1.4) \quad -\log \mathcal{L}(x) = -\sum_{j=1}^N \log \sigma(\varphi(a_j; x))_{y_j}.$$

If we now use the definition of the softmax function (1.3), we can write

$$\begin{aligned} -\log(\sigma(\varphi(a_j; x))_{y_j}) &= -\log\left(\frac{\exp(\varphi(a_j; x)_{y_j})}{\sum_{l=0}^9 \exp(\varphi(a_j; x)_l)}\right) \\ &= \log\left(\sum_{l=0}^9 \exp(\varphi(a_j; x)_l)\right) - \varphi(a_j; x)_{y_j}. \end{aligned}$$

The last expression is identical to the loss function in (1.2), when substituting $\varphi(a_j; x)_i$ with s_i . Therefore, minimizing (1.4) yields the same result as solving our optimization problem in (1.1). Also one can see that the choice of the softmax function to get the probabilities from the scores is crucial for the choice of our loss function.

1.3. Solvability.

The given optimization problem is a convex problem. This is because the log-sum-exp function is convex as we have already proved in the lecture. For reference one can also check [Boy09, pp. 72, 74]. Also the second term of our loss function (1.2) is linear and therefore convex. Also, φ is an affine mapping in x . And at the same time a convex function over an affine function is also convex (cf. [Boy09, p. 67]).

In general however, the optimization problem is not necessarily solvable. This is because the loss function (1.2) is not strictly convex in its scores s and therefore not strictly convex in the parameter x .

This, is because the loss function (1.2) is invariant under the addition of the same constant to every score s_i . More precisely, for a constant $c \in \mathbb{R}$ we have

$$\log\left(\sum_{l=0}^9 \exp(s_l + c)\right) = \log\left(\exp(c) \sum_{l=0}^9 \exp(s_l)\right) = \log\left(\sum_{l=0}^9 \exp(s_l)\right) + c$$

and also

$$\sum_{i=0}^9 p_i(s_i + c) = \sum_{i=0}^9 p_i s_i + c.$$

Therefore, we have

$$L(p, s + c\mathbb{1}) = \log\left(\sum_{l=0}^9 \exp(s_l + c)\right) - \sum_{i=0}^9 p_i(s_i + c) = L(p, s) + c - c = L(p, s).$$

This implies that for a solution x of our optimization problem, we get arbitrary additional solutions, e.g. $x' = (w_i + c\mathbb{1}, b_i)_{i=0}^9$ or $x' = (w_i, b_i + c)_{i=0}^9$ for any constant $c \in \mathbb{R}$. Therefore, the optimization problem is not uniquely solvable.

Even if we ignore non-uniqueness, the minimum in general may fail to be attained at all. A reason is linear separability of the training data. For a parameter x and

$$s_{j,i} := \varphi(a_j; x)_i = a_j^\top w_i + b_i,$$

and $p_{j,y_j} = 1$, we can rewrite the loss as

$$(1.5) \quad L(p_j, s_j) = \log \left(\sum_{l=0}^9 e^{s_{j,l}} \right) - s_{j,y_j} = \log \left(1 + \sum_{i \neq y_j} e^{-(s_{j,y_j} - s_{j,i})} \right) \geq 0.$$

Hence $L(p_j, s_j)$ gets small when all the $s_{j,y_j} - s_{j,i}$ are large, e.g. when the correct class score s_{j,y_j} dominates all other scores $s_{j,i}$ by a large margin $\gamma > 0$. If there exists a parameter $\bar{x} = (\bar{w}_i, \bar{b}_i)_{i=0}^9$ such that this is the case, we call the dataset *separable*.

We are then able to scale the parameter $\bar{x}$ with a factor $t > 0$

$$\bar{x}^{(t)} := (t\bar{w}_i, t\bar{b}_i)_{i=0}^9.$$

and get $s_{j,i}^{(t)} = \varphi(a_j; x^{(t)})_i = t s_{j,i}$. Thus, for the differences we then get

$$s_{j,y_j}^{(t)} - s_{j,i}^{(t)} = t(s_{j,y_j} - s_{j,i}) \geq t\gamma.$$

If we use this, the above expression (1.5) yields

$$0 \leq L(p_j, \varphi(a_j; \bar{x}^{(t)})) = \log \left(1 + \sum_{i \neq y_j} e^{-t(s_{j,y_j} - s_{j,i})} \right) \leq \log(1 + 9e^{-t\gamma}) \xrightarrow{t \rightarrow \infty} 0.$$

Therefore, if there exists such a separating parameter $\bar{x}$, the optimization term $\sum_{j=1}^N L(p_j, \varphi(a_j; \bar{x}^{(t)}))$ is going to be arbitrarily small for large t , i.e. for large weights and biases $t\bar{w}_i$ and $t\bar{b}_i$. This implies that the minimum of our optimization problem is not attained at a finite parameter x and therefore the optimization problem is not solvable.

Remark 1.3.1. The idea, that linear separability can be a problem for logistic regression, was inspired by [Ada18, Linear Separability and Regularization].

1.4. Regularization.

In order to deal with the problem of linear separability, we can add a regularization term to our optimization problem. For example, we can add an L^2 regularization term $\frac{\lambda}{2} \sum_{i=0}^9 \|w_i\|^2$ with a regularization parameter $\lambda > 0$. This term penalizes large weights w_i and therefore prevents the optimization problem from being unbounded, e.g. in the case of linear separability.

The optimization problem then becomes

$$\underset{x \in (\mathbb{R}^{64 \times \mathbb{R}})^{10}}{\text{minimize}} \quad \frac{1}{N} \sum_{j=1}^N L(p_j, \varphi(a_j; x)) + \frac{\lambda}{2} \sum_{i=0}^9 \|w_i\|^2.$$

This optimization problem is now both coercive and strictly convex for fixed biases b_i and therefore uniquely solvable. But it is not strictly convex in the biases b_i and therefore not uniquely solvable in general.

One could now try to add an additional regularization term for the biases b_i to get a strictly convex optimization problem and therefore a uniquely solvable optimization problem for

fixed biases b_i . Also a different idea could be to add a constraint to the optimization problem, e.g. $\sum_{i=0}^9 b_i = 0$, to get rid of the invariance under addition of a constant to all scores s_i as described above. However, we will not go into details here and just stick to the L^2 regularization of the weights w_i .

2. STOCHASTIC GRADIENT DESCENT

In the previous section we formulated the training of the multi-class logistic regression model as the finite-sum optimization problem

$$(2.1) \quad \underset{x \in (\mathbb{R}^{64} \times \mathbb{R})^{10}}{\text{minimize}} \quad f(x) := \frac{1}{N} \sum_{j=1}^N f_j(x), \quad f_j(x) := L(p_j, \varphi(a_j; x)).$$

Such objectives are standard in statistical learning, since each term corresponds to the loss contributed by one training example. We solve (2.1) using gradient-based methods and focus on stochastic gradient descent (SGD), which leverages the finite-sum structure by replacing the full gradient with inexpensive stochastic estimates.[Bot12]

2.1. Gradient-based methods: cost–accuracy trade-offs.

The gradient of (2.1) decomposes as

$$(2.2) \quad \nabla f(x) = \frac{1}{N} \sum_{j=1}^N \nabla f_j(x).$$

This admits several gradient-based strategies whose performance is governed by the trade-off between *per-iteration cost* and the *quality* of the search direction.

2.1.1. Deterministic first-order method: full-batch gradient descent.

Full-batch gradient descent uses the exact gradient (2.2):

$$(2.3) \quad x^{k+1} = x^k - \eta_k \nabla f(x^k).$$

Its main advantage is that the update direction is deterministic and consistent across iterations: for sufficiently small step sizes (e.g. $\eta_k \leq 1/L$ when f is L -smooth), one obtains a guaranteed decrease of the objective and stable convergence behavior.[Boy09; Bub15] In convex settings, full-batch gradient descent achieves the standard $O(1/k)$ convergence rate in function value for smooth objectives.[Bub15]

The drawback is computational. Each iteration requires one full pass over all N samples to evaluate $\nabla f(x^k)$, so the per-iteration cost scales linearly with N and can be dominated by memory access and data movement.[Boy09] As a consequence, while the method is sample-efficient (each update uses all available information), it can be time-inefficient for large datasets: progress per iteration can be good, yet progress per unit wall-clock time may be poor when N is large.

A second practical issue is that the method can be sensitive to ill-conditioning: when the problem is poorly conditioned, the step size required for stability can be small, leading to slow progress unless one uses preconditioning or curvature information (e.g. quasi-Newton methods).

2.1.2. Stochastic first-order methods (SGD and mini-batches).

SGD replaces $\nabla f(x^k)$ by a stochastic estimator computed from a mini-batch $\mathcal{B}_k$ of size B :

$$(2.4) \quad g_k = \frac{1}{B} \sum_{j \in \mathcal{B}_k} \nabla f_j(x^k), \quad x^{k+1} = x^k - \eta_k g_k.$$

Under uniform sampling, g_k is an unbiased estimator of the full gradient, i.e. $\mathbb{E}[g_k \mid x^k] = \nabla f(x^k)$, and its variance typically decreases as B increases.[Bot12; Bub15] The key trade-off is that SGD substantially reduces the computational cost per update (from N gradient

contributions to B), but introduces gradient noise, which affects stability and asymptotic accuracy.

From an optimization viewpoint, the presence of noise changes the convergence picture. With diminishing step sizes (e.g. $\eta_k \propto 1/\sqrt{k}$ under bounded variance), SGD converges in expectation with a sublinear rate, and with constant step sizes it typically converges only to a neighborhood whose radius depends on the step size and the gradient variance.[Bub15; Bot12] In practice this motivates using step-size schedules (constant during an initial phase and then decreasing) and monitoring performance as a function of *epochs* (passes over the data) rather than iterations alone.[Bot12]

Mini-batches provide an additional lever. Small B yields very cheap, frequent updates but high variance and potentially oscillatory behavior; larger B reduces variance and makes the method behave closer to full-batch gradient descent, but increases the cost per update.[Bot12] Importantly, increasing B does not always translate into proportional speedups in wall-clock time: beyond a certain regime, one may observe diminishing returns because the noise is already sufficiently reduced and additional samples mainly add redundant computation.[SLASFD19] Thus, B should be viewed as a computational hyperparameter that trades off hardware efficiency (vectorization/throughput) against statistical efficiency (noise reduction).[Bot12; SLASFD19]

Overall, full-batch GD tends to be preferable when N is moderate and high-accuracy solutions are required, whereas SGD is often preferable for large-scale training where the objective should decrease quickly in wall-clock time and moderate noise can be tolerated.[Bot12; Bub15]

3. IMPLEMENTATION

This section describes the software organization and training pipeline used to solve the handwritten digit classification task. The mathematical formulation of the model, loss function, and gradients is given in Section 1. Here we focus on the code structure, data processing, optimization routines (SGD and full-batch GD), and the experimental setup used for hyperparameter tuning and evaluation.

3.1. Code structure.

The implementation is split into the following files:

- *data.py*: dataset loading, normalization, and train/validation/test splitting.
- *model.py*: model definition (weights/biases), numerically stable softmax and loss, and analytical gradients.
- *sgd.py*: mini-batch SGD routine, logging, and threshold-based early stopping.
- *full_gd.py*: full-batch gradient descent baseline with the same objective.
- *metrics.py*: accuracy and auxiliary timing utilities.
- *experiments.py*: hyperparameter sweeps (L2, learning rate, batch size), selection of best run, and final training.
- *plots.py*: plotting utilities for objectives, validation accuracy, confusion matrices, and optimality gaps.
- *main.py*: orchestration script running the full experimental pipeline and producing figures/reports.

3.2. Data pipeline.

We use *sklearn.datasets.load_digits*. In this dataset, each feature corresponds to a grayscale pixel intensity stored as an integer in the range $[0, 16]$. We rescale the inputs with $X \leftarrow X/16$ to map values to $[0, 1]$, which is a standard feature-scaling step to improve numerical conditioning and optimization behavior in gradient-based methods. [GBC16, Ch. 12.2.1] We split the dataset into train/validation/test with stratified splits (i.e., each split keeps approximately the same class distribution as the full dataset) using *sklearn.model_selection.train_test_split* with the *stratify* option enabled. The default split is:

$$\text{train} = 0.70, \quad \text{val} = 0.15, \quad \text{test} = 0.15,$$

with a fixed *random_state* for reproducibility.

3.3. Model, objective, and derivatives.

The multi-class linear predictor is implemented as

$$S = XW + \mathbf{1}b^\top, \quad W \in \mathbb{R}^{64 \times 10}, \quad b \in \mathbb{R}^{10},$$

where each column of W corresponds to one class.

Given scores $S \in \mathbb{R}^{n \times 10}$ and integer labels $y \in \{0, \dots, 9\}^n$, the mean cross-entropy is computed via a numerically stable *logsumexp*:

$$\ell(W, b) = \frac{1}{n} \sum_{j=1}^n \left(\log \sum_{i=0}^9 e^{S_{j,i}} - S_{j,y_j} \right).$$

When L2 regularization is enabled, the training objective becomes

$$f(W, b) = \ell(W, b) + \frac{\lambda}{2} \|W\|_F^2,$$

which is implemented by adding the penalty to the reported objective and applying weight decay in the gradient:

$$\nabla_W f = \nabla_W \ell + \lambda W, \quad \nabla_b f = \nabla_b \ell.$$

Probabilities are computed with a stable row-wise softmax,

$$P_{j,i} = \frac{e^{S_{j,i}}}{\sum_k e^{S_{j,k}}},$$

and we form $D = (P - Y)/n$, where Y is the one-hot encoding of labels. The gradients then follow:

$$\nabla_W \ell = X^\top D, \quad \nabla_b \ell = \sum_{j=1}^n D_{j,:}.$$

This is implemented in *gradients()* and reused by both SGD and full-batch GD.

3.4. Optimization algorithms.

3.4.1. Mini-batch SGD.

The function *train_sgd* implements epoch-based mini-batch SGD with optional shuffling. For each epoch, we iterate over mini-batches of size B and update

$$W \leftarrow W - \alpha \nabla_W f_{\mathcal{B}}(W, b), \quad b \leftarrow b - \alpha \nabla_b f_{\mathcal{B}}(W, b),$$

where $f_{\mathcal{B}}$ denotes the mini-batch objective (cross-entropy on the batch plus the optional L2 term).

During training, the implementation records the *full* training objective $f(W, b)$ (evaluated on the entire training set), wall-clock timestamps via *time.perf_counter*, and validation accuracy when validation data is available. To make runs comparable across hyperparameters, these quantities are logged every *eval_every* epochs.

Optionally, we enable a threshold-based stopping rule on validation accuracy: every *eval_every* epochs we compute acc_{val} and stop once it remains above *target_acc* for *target_patience* consecutive logged evaluations. Otherwise, training runs up to *max_epochs*.

3.4.2. Full-batch Gradient Descent baseline.

The baseline *train_full_gd* uses the same objective and gradients, but updates at each iteration using the full dataset:

$$W \leftarrow W - \alpha \nabla_W f(W, b), \quad b \leftarrow b - \alpha \nabla_b f(W, b).$$

It logs objective values and (optionally) validation accuracy after every iteration. When validation data is available, the same threshold-based rule can be applied: the run stops once $\text{acc}_{\text{val}} \geq \text{target_acc}$ for *target_patience* consecutive evaluations; otherwise it runs up to *max_iters*.

3.5. Experimental protocol.

The experimental workflow in *main.py* follows:

- (1) Tune λ (L2) for SGD with fixed baseline (α, B) using a grid over *reg_values*.
- (2) Tune learning rate α for SGD using a grid over *learning_rates*, keeping best λ .
- (3) Tune batch size B for SGD using a grid over *batch_sizes*, keeping best (λ, α) .
- (4) Final SGD training on train+val, then evaluate on test.
- (5) Tune learning rate for full-batch GD (same λ), then final GD training on train+val and evaluate on test.

Each sweep returns a list of *RunResult* objects containing: final model, objective history, wall-clock time, and accuracies. The helper *pick_best* selects the best configuration (by default maximizing validation accuracy).

3.6. Evaluation metrics and visualizations.

Classification accuracy is computed as

$$\text{acc} = \frac{1}{n} \sum_{j=1}^n \mathbb{I}(\hat{y}_j = y_j).$$

In addition, *main.py* prints a per-class report via *sklearn.metrics.classification_report*.

The following figures are generated:

- objective value vs. logged steps (for hyperparameter sweeps),
- validation accuracy vs. time (to compare convergence speed),
- validation accuracy vs. hyperparameter values (λ , α , B),
- confusion matrices on the test set (SGD vs. full-batch GD).

To compare SGD against a reference solution, we obtain an empirical reference value $f^\star$ by running long fixed-step full-batch GD and taking the minimum objective value observed along the trajectory. We then plot the optimality gap

$$f(x_k) - f^\star$$

on a log scale as a function of effective data passes (SGD logs every *eval_every* epochs).

4. RESULTS AND DISCUSSION

In this section we present some of the results of our experiments and discuss their implications. We focus on the comparison between full-batch gradient descent (FGD) and stochastic gradient descent (SGD) in terms of convergence behavior and computational efficiency. Also we analyze the effects of different hyperparameters, such as the learning rate, the mini-batch size and the L^2 regularization parameter on the performance of SGD.

For evaluation, we use three disjoint subsets with the default proportions

$$\text{train} = 0.70, \quad \text{val} = 0.15, \quad \text{test} = 0.15.$$

The split is computed with `sklearn.model_selection.train_test_split` using the `stratify` option, so that each subset preserves (approximately) the original class proportions.

4.1. Hyperparameter tuning.

In order to find good hyperparameters for our models, we followed the approach to tune them sequentially on the validation set. We are aware that this approach is not optimal, since the hyperparameters are not independent of each other and therefore it would be better to tune them jointly, e.g. by doing a grid search over the hyperparameter space. However we decided to follow this approach for simplicity.

4.1.1. L^2 regularization.

To select the L^2 regularization strength, we ran SGD with fixed $\alpha = 0.5$ and $B = 64$ over a grid of λ values and compared the resulting validation accuracy (Figure 2). The effect of regularization is clearly visible: accuracy remains essentially unchanged for very small λ , while it deteriorates as λ becomes larger (around 10^{-4} – 10^{-2}). This behavior is consistent with the bias–variance trade-off: moderate regularization can help control overfitting, but overly large λ penalizes the weights too strongly, leading to underfitting and reduced validation performance [GBC16, Ch. 5.4.4]. In our experiments, the best results are obtained with λ close to zero (or very small values), suggesting that the unregularized model already generalizes well on this dataset under the chosen training setup.

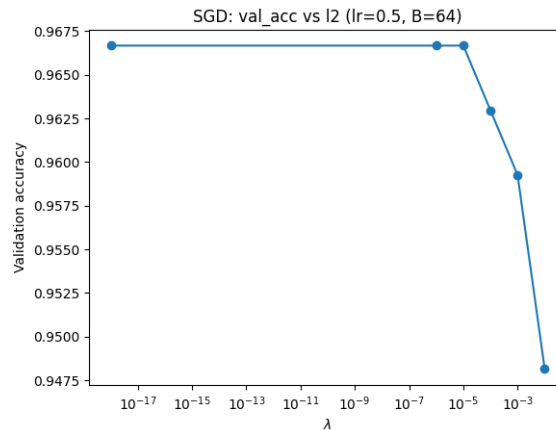


FIGURE 1. Final validation accuracy vs. L^2 regularization parameter for SGD

4.1.2. Learning rate.

We also studied the effect of the learning rate on convergence speed by comparing validation accuracy as a function of wall-clock time for both SGD and full-batch GD. In these

runs, we fixed the regularization strength to $\lambda = 0$ and, for SGD, we fixed the batch size to $B = 64$.

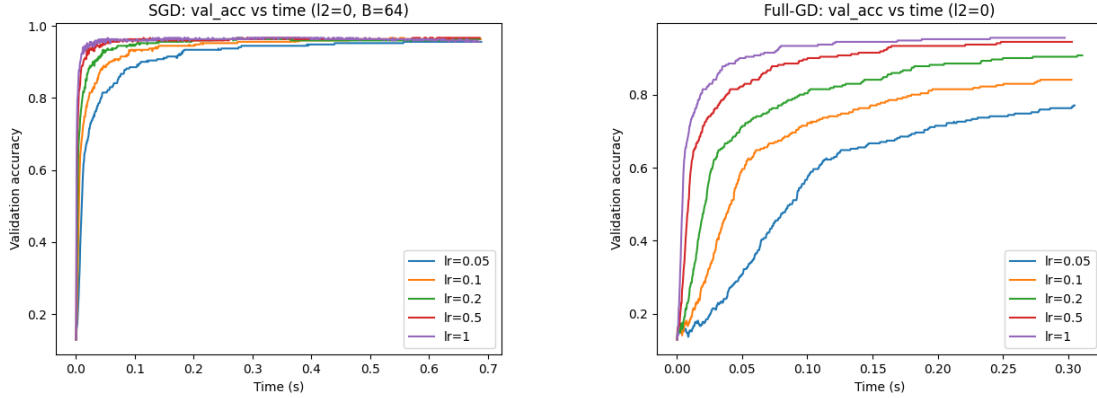


FIGURE 2. Validation accuracy vs. time for different learning rates in SGD (left) and full-batch GD (right).

As expected, the learning rate strongly affects the convergence behavior of both methods: smaller values yield slower progress, while larger values reach high accuracy in fewer seconds (up to the stability limit) [GBC16, Ch. 11.4.1]. In this setup, SGD reaches a high validation accuracy substantially faster than full-batch GD, while achieving comparable final performance.

4.1.3. Batch size.

We next investigated the effect of the mini-batch size B on the validation accuracy of SGD, keeping $\lambda = 0$ and $\alpha = 0.5$ fixed (Figure 3). The batch size influences both the stochasticity of the gradient estimate and the number of parameter updates performed per epoch (approximately n/B), so the comparison reflects both optimization dynamics and the imposed training budget.

In our runs, intermediate batch sizes performed best: accuracy improves from very small batches up to $B \approx 64$ –128, while larger batches show a slight degradation. This is consistent with the usual trade-off between noisy but frequent updates for small B and smoother but less frequent updates for large B , under a fixed epoch budget. [GBC16, Ch. 8.1.3]

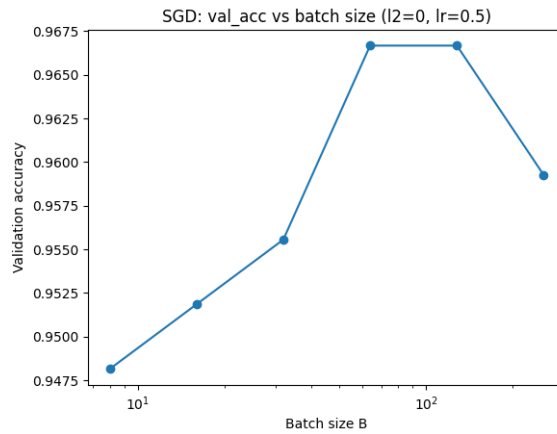


FIGURE 3. Validation accuracy vs. batch size for SGD ($\lambda = 0$, $\alpha = 0.5$).

For this sweep we used a fixed maximum number of epochs and enabled threshold stopping when the validation accuracy reached 0.96 (with the configured patience). The best configuration in this grid is $B = 64$ (Figure 3).

4.2. Final models and discussion.

We finally compare the two selected configurations (SGD and full-batch GD) after training on the combined train+validation set, and report performance on the held-out test set. The confusion matrices in Figure 4 show that both methods achieve strong test performance, with most mass concentrated on the diagonal and only a small number of misclassifications.

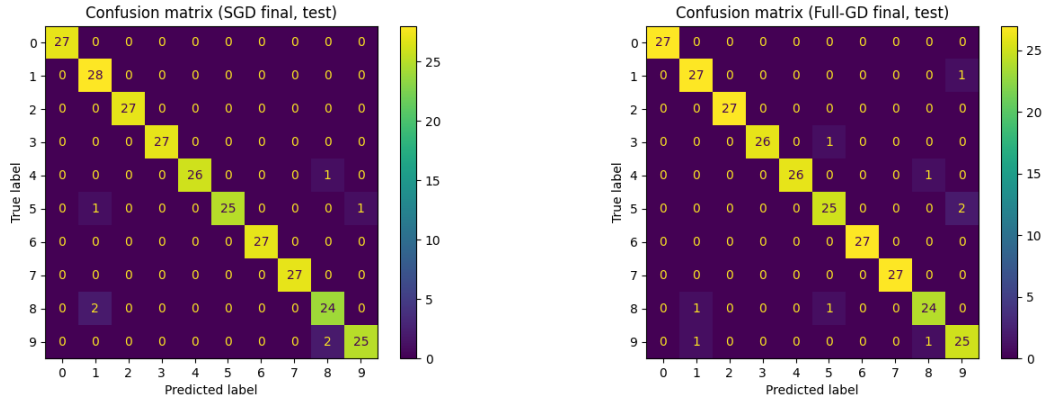


FIGURE 4. Confusion matrices on the test set for the final SGD model (left) and the final full-batch GD model (right).

To compare optimization behavior, Figure 5 reports the objective value and the corresponding optimality gap as a function of effective data passes. In this experiment, SGD reduces the objective more rapidly during the initial phase, which is consistent with its frequent parameter updates based on mini-batches. Full-batch GD follows a smoother trajectory because it uses the exact full gradient at every step, but each iteration is computationally heavier.

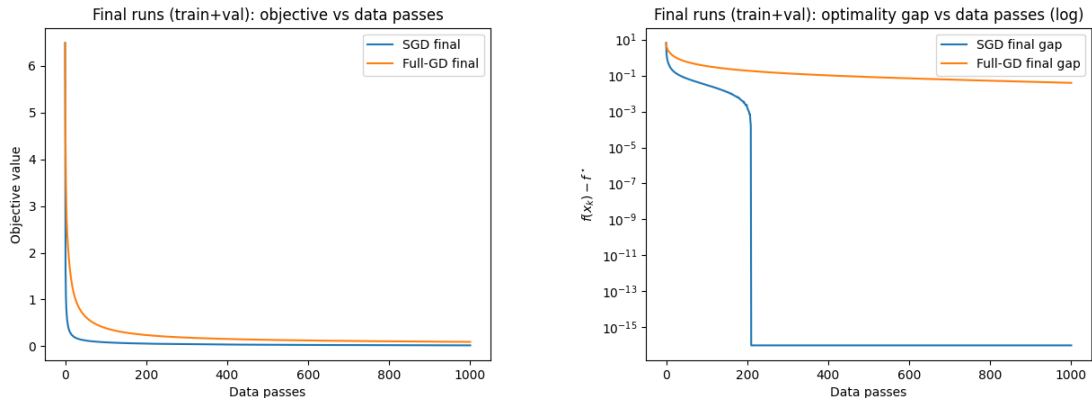


FIGURE 5. Objective value (left) and optimality gap (right, log scale) vs. data passes for the final SGD and full-batch GD runs (trained on train+val).

In the optimality-gap plot, the reference value f^* is computed empirically as the minimum objective observed along a long fixed-step full-batch GD run. This value should therefore

be interpreted as a numerical baseline rather than a certified optimum. In our experiments, the final SGD run achieves a lower training objective than this reference, i.e. $f_{\text{SGD}} < f^*$, indicating that the chosen reference run does not attain the best objective value encountered in our study. Consequently, the plotted “gap” mainly reflects proximity to this baseline and is sensitive to how the reference f^* is constructed.

Overall, for problems where full gradients are expensive (large n and/or high-dimensional models), SGD (or mini-batch variants) is typically preferred due to its lower per-update cost and favorable time-to-accuracy. Full-batch GD can be attractive when the dataset is small enough that full gradients are cheap, since its updates are deterministic and its objective trajectory is smoother.

REFERENCES

- [Ada18] Ryan Adams. *Linear Classification with Logistic Regression. Lecture notes*. COS 324 – Elements of Machine Learning. Princeton University. Oct. 8, 2018. URL: <https://www.cs.princeton.edu/courses/archive/spring19/cos324/files/logistic-regression.pdf> (visited on 02/10/2026).
- [Bot12] Léon Bottou. “Stochastic Gradient Descent Tricks”. In: *Neural Networks: Tricks of the Trade*. Ed. by Grégoire Montavon, Geneviève B. Orr, and Klaus-Robert Müller. Vol. 7700. Lecture Notes in Computer Science. Springer, 2012, pp. 421–436.
- [Boy09] Stephen P. Boyd. *Convex optimization*. Ed. by Lieven Vandenbergh. 7th ed. first published 2004. Cambridge University Press, 2009. 716 pp. ISBN: 9780521833783.
- [Bub15] Sébastien Bubeck. *Convex Optimization: Algorithms and Complexity*. Foundations and Trends® in Machine Learning. FnT in Machine Learning, 8(3-4):231–357. 2015.
- [Dev] scikit-learn Developers. *Documentation of scikit-learn Machine Learning in Python*. Version 1.8.0 (stable). URL: <https://scikit-learn.org> (visited on 02/09/2026).
- [GBC16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [SLASFD19] Christopher J. Shallue, Jaehoon Lee, Joseph Antognini, Jascha Sohl-Dickstein, Roy Frostig, and George E. Dahl. “Measuring the Effects of Data Parallelism on Neural Network Training”. In: *Journal of Machine Learning Research* 20.112 (2019), pp. 1–49.